LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASICS WITH COMPOSE

Oleh:

Faisal Tanjung NIM. 2410817310012

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman I Modul 1: Android Basics With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman I. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Faisal Tanjung
NIM : 2410817310012

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom NIP. 19971031 202506 1 009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 9
B. Output Program 14
C. Pembahasan 16
D. Tautan Git 21

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 6
Gambar 2. Tampilan Dadu Setelah di-Roll 7
Gambar 3. Tampilan Dadu Roll Double 8
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose 14
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose 14
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose 15
Gambar 7. Screenshot Hasil Jawaban Soal 1 XML 15
Gambar 8. Screenshot Hasil Jawaban Soal 1 XML 16
Gambar 9. Screenshot Hasil Jawaban Soal 1 XML 16

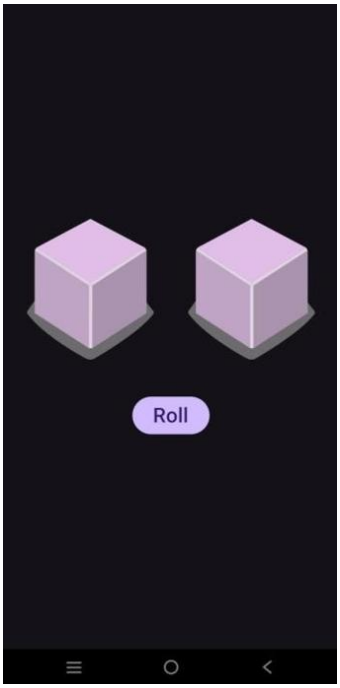
DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Compose Soal 1 11
Tabel 2. Source Code MainActivity.kt XML Soal 1 13
Tabel 3. Source Code activity_main.xml Soal 1 14

SOAL 1

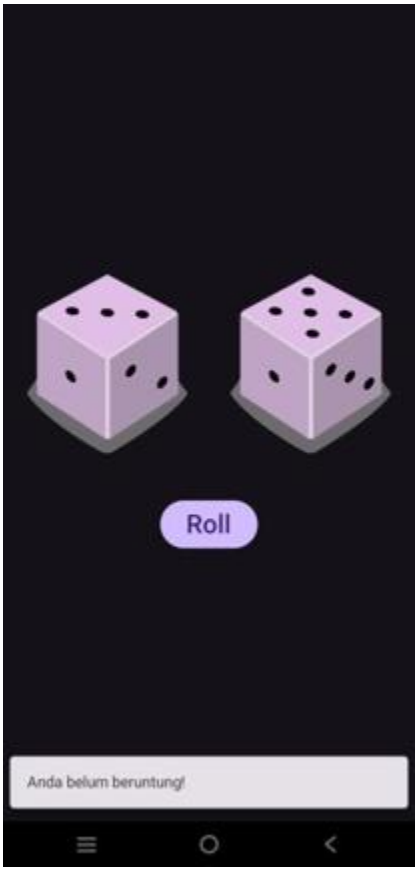
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



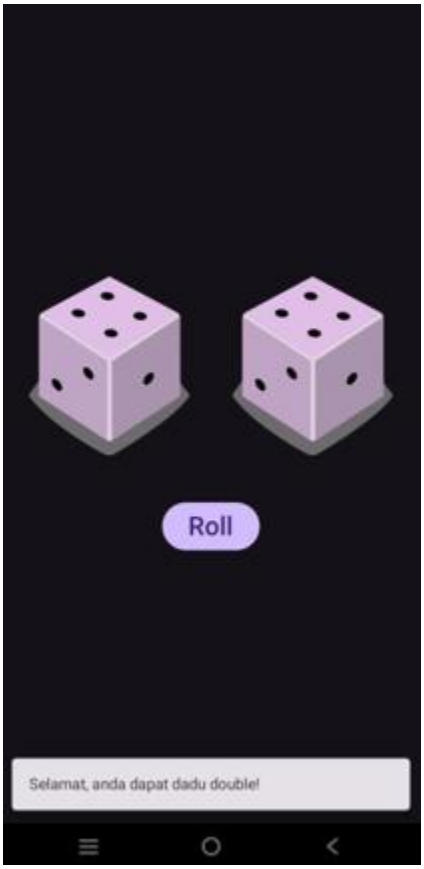
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan "Selamat, anda dapat dadu double!" seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Dadu Roll Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut:

   https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

Additional:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan.

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java?
3. Sebutkan, dan jelaskan mengenai SOLID Principles
4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

A. Source Code

```
package com.example.dicerolller

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import
com.example.dicerolller.ui.theme.DiceRolllerTheme
import
androidx.compose.foundation.layout.wrapContentSize
import androidx.compose.ui.Alignment
import androidx.compose.foundation.layout.Column
import androidx.compose.material3.Button
import androidx.compose.ui.res.stringResource
import androidx.compose.foundation.Image
import androidx.compose.ui.res.painterResource
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.Spacer
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.ui.unit.dp
import androidx.compose.runtime.remember
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.foundation.layout.Row
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.material3.SnackbarDuration
import androidx.compose.runtime.rememberCoroutineScope
import kotlinx.coroutines.launch

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?)
{
        super.onCreate(savedInstanceState)
```

```
        setContent {
            DiceRolllerTheme {
                DiceRollerApp()
            }
        }
    }
}

@Preview
@Composable
fun DiceRollerApp(){
    DiceWithButtonAndImage(modifier = Modifier
        .fillMaxSize()
        .wrapContentSize(Alignment.Center)

    )
}

@Composable
fun DiceWithButtonAndImage(modifier: Modifier =
Modifier){
    var result1 by remember { mutableIntStateOf(0) }
    var result2 by remember { mutableIntStateOf(0) }
    val snackbarHostState = remember {
SnackbarHostState() }
    val scope = rememberCoroutineScope()

    fun getImageResource (result: Int): Int {
        return when (result){
            0 -> R.drawable.dice_0
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }
    }

    Scaffold(
        snackbarHost = { SnackbarHost(hostState =
snackbarHostState) }
    ) { paddingValues ->
        Column(
            modifier = modifier.padding(paddingValues),
            horizontalAlignment =
Alignment.CenterHorizontally
```

```
        ) {
            Row(
                verticalAlignment =
Alignment.CenterVertically
            ) {
                Image(
                    painter =
painterResource(getImageResource(result1)),
                    contentDescription =
result1.toString()
                )
                Image(
                    painter =
painterResource(getImageResource(result2)),
                    contentDescription =
result2.toString()
                )
            }

            Spacer(modifier = Modifier.height(24.dp))

            Button(onClick = {
                result1 = (1..6).random()
                result2 = (1..6).random()

                scope.launch {
                    if (result1 == result2) {
                        snackbarHostState.showSnackbar(
                            message = "Selamat, anda
dapat dadu double!",
                            duration =
SnackbarDuration.Short
                        )
                    } else {

snackbarHostState.showSnackbar("Anda belum beruntung!")
                    }
                }
            }) {
                Text(stringResource(R.string.roll))
            }
        }
    }
}

```

Tabel 1. Source Code MainActivity.kt Compose Soal 1

```
1  package com.example.diceroller
2
3  import android.os.Bundle
4  import android.widget.Button
5  import android.widget.ImageView
6  import androidx.appcompat.app.AppCompatActivity
7  import com.google.android.material.snackbar.Snackbar
8
9  class MainActivity : AppCompatActivity() {
10
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         setContentView(R.layout.activity_main)
14         val rollButton: Button =
15 findViewById(R.id.roll_button)
16         val diceImage1: ImageView =
17 findViewById(R.id.dice_image1)
18         val diceImage2: ImageView =
19 findViewById(R.id.dice_image2)
20         val rootLayout =
21 findViewById<android.view.View>(R.id.root_layout)
22
23         rollButton.setOnClickListener {
24             val result1 = (1..6).random()
25             val result2 = (1..6).random()
26
27
28 diceImage1.setImageResource(getDiceResource(result1))
29
30 diceImage2.setImageResource(getDiceResource(result2))
31
32             if (result1 == result2) {
33                 Snackbar.make(rootLayout, "Selamat, anda
34 dapat dadu double!", Snackbar.LENGTH_SHORT).show()
35             } else {
36                 Snackbar.make(rootLayout, "Anda belum
37 beruntung!", Snackbar.LENGTH_SHORT).show()
38             }
39         }
40     }
41
42     private fun getDiceResource(result: Int): Int {
43         return when (result) {
44             1 -> R.drawable.dice_1
45             2 -> R.drawable.dice_2
46             3 -> R.drawable.dice_3
47             4 -> R.drawable.dice_4
```

```
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }
    }
}

```

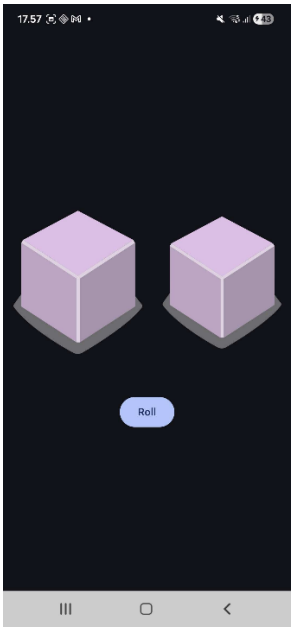
Tabel 2. Source Code MainActivity.kt XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout

xmlns:android="http://schemas.android.com/apk/res/andro
id"
    android:id="@+id/root_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="16.dp">

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <ImageView
            android:id="@+id/dice_image1"
            android:layout_width="160dp"
            android:layout_height="160dp"
            android:src="@drawable/dice_0" /> <View
        android:layout_width="16dp"
        android:layout_height="0dp" />

        <ImageView
            android:id="@+id/dice_image2"
            android:layout_width="160dp"
            android:layout_height="160dp"
            android:src="@drawable/dice_0" />
    </LinearLayout>

    <Button
        android:id="@+id/roll_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"

```

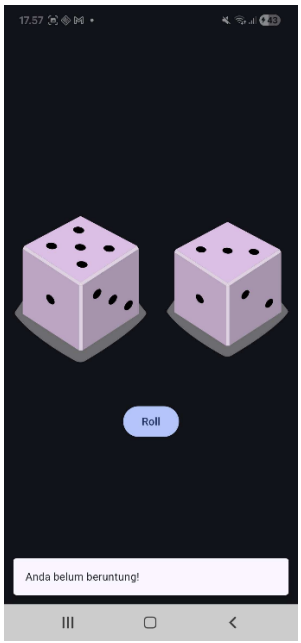
```
        android:text="@string/roll" />
</LinearLayout>

```

Tabel 3. Source Code activity_main.xml Soal 1

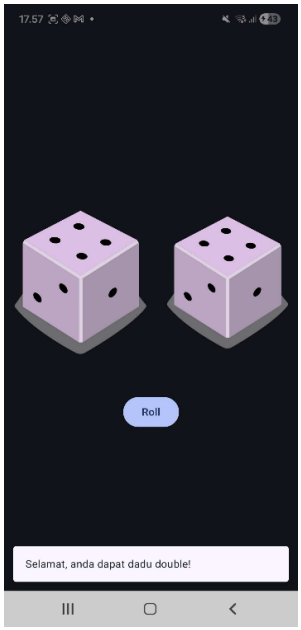
B. Output Program



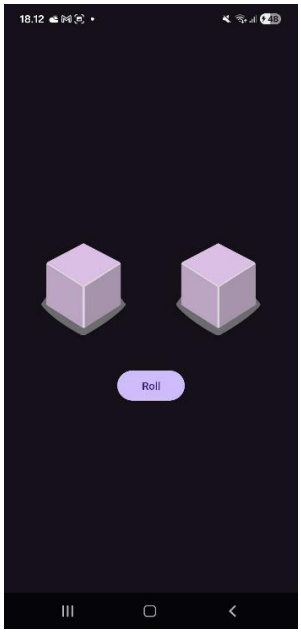
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose



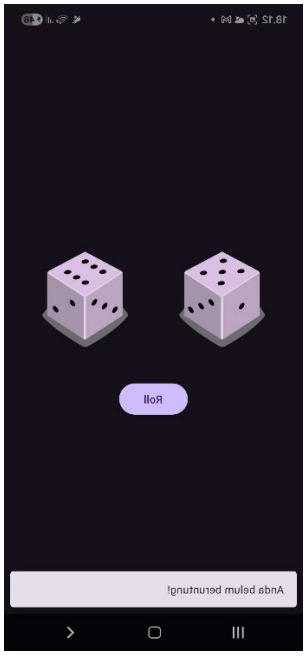
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose



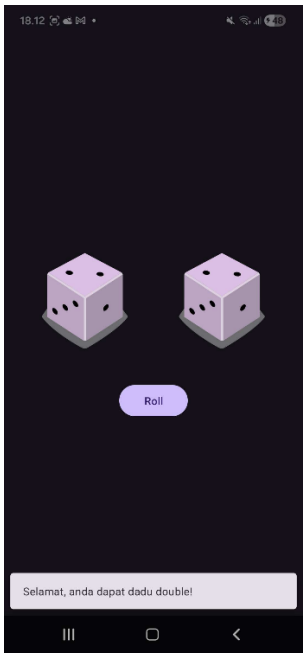
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 7. Screenshot Hasil Jawaban Soal 1 XML



Gambar 8. Screenshot Hasil Jawaban Soal 1 XML



Gambar 9. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

File `MainActivity.kt` **Compose** merupakan Aplikasi sederhana yang mengimplementasikan UI Toolkit Jetpack Compose dengan menampilkan dua dadu dan

terjadi perubahan ketika menekan tombol roll. Fungsi `onCreate()` merupakan sebuah entry atau pintu masuk utama pada aplikasi ini. Ketika pertama kali membuka aplikasi, android secara otomatis memanggil `onCreate()`. Didalamnya terdapat `setContent{}` yang berfungsi memasang tampilan compose secara keseluruhan ke layar monitor. Tanpa adanya fungsi ini, aplikasi tidak bisa menampilkan apapun karena tidak ada UI yang terpasang ke activity. Selanjutnya ada fungsi `DiceRollerApp()` yang berguna sebagai titik awal UI untuk menjembatani antara activity dan komponen tampilan utama. Tugasnya yaitu memanggil `DiceWithButtonAndImage()` sambil memberi instruksi layout berupa modifier agar seluruh konten mengisi layar penuh dan berada tepat di Tengah. Selain itu fungsi ini juga memiliki anotasi `@preview` agar developer dapat melihat tampilan UI langsung pada Android Studio tanpa harus melakukan running pada emulator terlebih dahulu. Kemudian fungsi `DiceWithButtonAndImage()` yang berfungsi untuk melakukan tiga hal utama, pertama yaitu mengelola state dari kedua dadu melalui variabel `result1` dan `result2` yang dimana akan memicu update secara otomatis pada tampilan dadu setiap kali nilainya berubah. Kedua yaitu membangun seluruh struktur tampilan mulai dari gambar dadu, tombol roll, hingga snackbar untuk menampilkan pesan. Ketiga yaitu menangani business logic dengan mengacak angka dadu dan memutuskan pesan apa yang akan ditampilkan nantinya. Selanjutnya fungsi `getImageResource()` berfungsi sebagai penerjemah antara angka yang dihasilkan dari acakan logika bisnis dengan gambar yang harus ditampilkan. Fungsi `onClick` pada button berfungsi untuk melakukan 3 hal ketika tombol roll ditekan, yaitu mengacak nilai variabel `result1` dan `result2` dengan rentang antara 1 sampai 6, lalu mengirimkan coroutine melalui `scope.launch{}`, dan coroutine tersebut dilakukan pengecekan apakah dadu yang akan keluar ini menghasilkan angka yang sama atau tidak agar pesan yang akan diberi itu sesuai. Lalu fungsi `remember{}` dan `mutableIntStateOf()` yang berfungsi sebagai sistem memori. Tanpa kedua fungsi ini, maka compose akan mengembalikan nilai variabel `result1` dan `result2` ke 0 setiap terjadi recomposition. Terakhir, fungsi `scope.launch{}` dan `showSnacbar()` berfungsi untuk menampilkan pesan notifikasi kepada pengguna.

File `activity_main.xml` **XML** merupakan bagian layout untuk aplikasi sederhana yang mengimplementasikan UI Toolkit XML. Pertama bagian LinearLayout

terdapat xmlns:android yang merupakan deklarasi namespace yang berada di elemen terluar berfungsi untuk memberitahu Android bahwa semua atribut berawalan android: berasal dari framework android. android:id="@+id/root_layout" berfungsi untuk mendaftarkan ID unique agar layout bisa diakses dari kode kotlin melalui findViewById().Atribut layout_width dan layout_height yang diisi match_parent membuat layout mengisi seluruh layar, sementara gravity="center" memusatkan semua konten di dalamnya. orientation="vertical" menyusun child view dari atas ke bawah, dan padding="16dp" memberi jarak antara tepi layout dengan kontennya. Perlu diperhatikan bahwa penulisan "16.dp" pada kode ini keliru karena itu adalah sintaks Kotlin Compose, penulisan yang benar di XML adalah "16dp". Kedua bagian Child dari LinearLayout yang bertugas untuk Menyusun dua gambar dadu secara bedampingan. Wrap_content pada lebar dan tinggi membuat ukuran layout hanya menyesuaikan content yang ada didalamnya, tidak lebih. orientation="horizontal" bertugas untuk memastikan kedua dadu tersusun dari kiri ke kanan. Bagian ketiga yaitu ImageView sebagai komponen untuk menampilkan gambar. Ukuran 160dp x 160dp memakai satuan dp agar tampilan konsisten di berbagai ukuran dan kepadatan layar. android:src="@drawable/dice_0" menentukan gambar awal yang ditampilkan sebelum tombol roll pertama kali ditekan. Keempat bagian View dengan lebar 16dp dan tinggi 0dp untuk memisahkan antara dua gambar dadu secara manual. Terakhir ada bagian Button yang berisi layout_marginTop="24dp" untuk memberi jarak antara tombol dengan elemen di atasnya. Berbeda dengan padding yang memberi jarak di dalam elemen, margin memberi jarak di luar elemen terhadap sekitarnya. android:text="@string/roll" mengambil teks dari file strings.xml alih-alih menulis teks langsung, ini adalah praktik terbaik Android agar aplikasi mudah dialih bahasakan tanpa mengubah file layout.

Terakhir yaitu file MainActivity.kt versi **XML** berisi logic – logic untuk menjalankan aplikasi ini dengan mengimplementasi UI Toolkit XML. Fungsi onCreate() bertugas sebagai entry point ketika aplikasi dibuka. Selain itu fungsi ini juga memanggil setContentView(R.layout.activity_main) untuk memasang file XML (layout) ke layar monitor. Lalu fungsi findViewById() yang berperan sebagai jembatan

antara kode kotlin dengan elemen – elemen yang di define di file XML (layout). `setOnClickListener {}` berfungsi mendaftarkan sebuah aksi yang akan dijalankan setiap tombol roll ditekan. Ketika ditekan, fungsi ini mengacak dua nilai dadu, memperbarui gambar, dan menampikan snackbar. Kemudian fungsi `setImageResource()` berfungsi mengganti gambar yang ditampilkan oleh ImageView ketika tombol roll ditekan. Ia menerima sebuah resource ID gambar yang didapatkan dari fungsi `getDiceResource()` dan langsung memperbarui tampilan gambar di layar.

Essay

Imperative Programming merupakan paradigma/pola pikir yang didefinisikan bagaimana hal – hal itu dilakukan, seperti menulis langkah demi langkah program fungsionalitas dan logika dengan menggunakan perintah. Declarative Programming merupakan paradigma yang didefinisikan sesuatu yang hanya mendeklarasikan data. Dengan ini, Declarative Programming dapat dikatakan sebagai paradigma tanpa fungsionalitas (Martelin, 2010) . Untuk pengimplementasian pada Imperative Programming, Seorang developer harus membuat komponen tombol dan menambahkan fungsi penanganan kejadian (event handler) ke dalamnya. Fungsi tersebut akan dijalankan saat diklik dan berisi pernyataan if-else yang mengubah warna berdasarkan warna tombol saat ditekan. Berbeda dengan Declarative Programming yaitu membuat developernya hanya perlu membuat komponen tombol dengan properti toggle color (Rootstrap, 2022). Menurut opini saya, paradigma Declarative Programming lebih efisien dan mudah digunakan karena bakal mengurangi beban kognitif para developer nya karena dapat meringkas kode lebih singkat dan memudahkan para developer dalam mengelola UI.

Dalam OOP, ketika developer ingin menambahkan fitur baru ke sebuah class, pendekatan yang digunakan adalah dengan membuat class turunan melalui inheritance. Kotlin menawarkan hal yang berbeda melalui fitur Extension Function, yaitu kemampuan untuk menambahkan fungsi baru ke class tanpa harus mewarisi class tersebut. Berbeda dengan Java yang tidak memiliki fitur yang seperti itu. Untuk melakukan hal yang sama, fitur yang disediakan Java yaitu wrapper class yang justru akan menambah kerumitan kode. Selain itu masih dalam OOP, penanganan error merupakan bagian penting dari desain sebuah class.

Bahasa Java menerapkan konsep checked exception yang dimana setiap method yang memiliki resiko melempar exception wajib menanganinya menggunakan try-catch atau mendeklarasikan throws. Sedangkan Kotlin memiliki pendekatan yang berbeda dengan tidak menggunakan checked exception sama sekali. Semua exception pada Kotlin merupakan turunan dari class Throwable yang tidak wajib untuk ditangani. Hal ini menguntungkan untuk Kotlin dikarenakan checked exception pada Java beresiko memutus alur logika kode (Banerjee, Bose, Kundu, & Mukherjee, 2018).

SOLID merupakan kepanjangan dari SRP (Single Responsibility Principle), OCP (Open Closed Principle), LSP (Liskov Subtitution Principle), ISP (Interface Segregation Principle), dan DIP (Depedency Inversion Principle). SRP menyatakan bahwa suatu class hanya memiliki satu tanggung jawab, bukan banyak. OCP menyatakan bahwa suatu class harus tertutup untuk modifikasi dan harus terbuka untuk ekstensi. LSP menyatakan bahwa bahwa seorang induk (parent) harus bisa menggantikan child dengan mudah. ISP menyatakan bahwa sebuah client tidak boleh dipaksa menggunakan interface jika tidak membutuhkannya. Terakhir, DIP menyatakan dua hal, yaitu sebuah modul Tingkat tinggi tidak boleh bergantung pada modul tingkat rendah, keduanya harus bergantung pada abstraksi. Kedua Abstraksi tidak boleh bergantung pada detail, Detail harus bergantung pada abstraksi (Madasu, Venna, & Eltaeib, 2015).

Class pada Bahasa Pemrograman Kotlin merupakan blueprint umum untuk membuat objek yang dimana developer mendeklarasikan sendiri semua behavior yang diinginkan secara manual. Ketika membuat sebuah Class, Kotlin tidak secara otomatis menyediakan fungsi – fungsi pendukung. Semuanya harus dideklarasikan sendiri. Sedangkan Data Class merupakan class khusus yang dirancang untuk menyimpan data. Class ini secara otomatis menggenerate beberapa fungsi penting di balik layar tanpa perlu di deklarasikan secara manual.

Daftar Pustaka

Martelin, T. (2010). Orientation awareness in declarative user interface languages for mobile devices: A case study and evaluation [Master's thesis, Aalto University]. Aalto University Digital Repository. URL:

https://aaltodoc.aalto.fi/server/api/core/bitstreams/ 065d713f-f6c8-4c6d-b7ed-f0fdfbccf5ee/content

Rootstrap. (2022, September 23). Imperative v declarative UI design: Is declarative programming the future? Rootstrap. URL: https://www.rootstrap.com/blog/imperative-v-declarative-ui-design- is-declarative-programming-the-future

Banerjee, M., Bose, S., Kundu, A., & Mukherjee, M. (2018). A comparative study: Java vs Kotlin programming in Android application development. *International Journal of Advanced Research in Computer Science*, 41-45.

Madasu, V. K., Venna, T. V., & Eltaeib, T. (2015). SOLID principles in software architecture and introduction to RESM concept in OOP. *Journal of Multidisciplinary Engineering Science and Technology*, 1-3.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/Lawdiy